A Comparative Regression Analysis of Machine Learning Models for Symbol Detection in Molecular Communication Channels

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Recent work has shown that neural networks (NNs) can be applied to symbol detection in molecular communication (MC) channels, and that their behavior can be explained using post-hoc methods such as LIME and individual conditional expectation (ICE), revealing an analogy to classical peak and slope detectors [1]. However, prior explainability studies have focused on a single classification model. This paper extends that line of work by framing MC channel response estimation as a regression problem and presenting a comparative analysis of several regression algorithms — Random Forest (RF), Support Vector Regression (SVR), Artificial Neural Networks (ANN), Gradient Boosting Regression (GBR), and Gaussian Process Regression (GPR) — trained on a synthetic dataset generated from the same channel model and parameter ranges as [1]. We evaluate each model on its ability to reconstruct the channel impulse response under low- and high-interference regimes.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Molecular communication (MC) research has increasingly relied on machine learning (ML) models for symbol detection, motivated by the lack of closed-form, tractable channel models and the difficulty of setting fixed detection thresholds under varying transmitter–receiver distances [1]. Neural network (NN)-based detectors have shown strong empirical performance, but their adoption for safety-critical applications such as in-vivo drug delivery and cancer cell detection has been limited by their black-box nature [1], [2]. Torres Gómez et al. [1] addressed this gap by applying the local interpretable model-agnostic explanation (LIME) method and individual conditional expectation (ICE) plots to a feed-forward NN classifier trained on synthetic and testbed-derived MC data, showing that the trained NN implicitly recovers classical peak- and slope-detection strategies depending on the interference regime. That study, however, considers a single NN architecture applied to a binary classification task. This paper extends [1] along two directions. First, we reformulate part of the problem as regression: rather than only classifying a received symbol as 0 or 1, we train models to predict the continuous channel response $N_R x(t)$, allowing direct comparison against the known closed-form channel impulse response (CIR). Second, we broaden the model comparison beyond a single NN, evaluating Random Forest (RF), Support Vector Regression (SVR), Gradient Boosting Regression (GBR), Gaussian Process Regression (GPR), and an ANN regressor under an identical synthetic dataset and evaluation protocol, using the same parameter ranges as [1][Table I].

Our contributions are as follows:

- We generate a synthetic regression dataset for MC symbol detection using the same channel model, parameter ranges, and low-/high-interference regimes as [1], enabling direct comparison with the base study.
- We train and compare five regression algorithms (RF, SVR, ANN, GBR, GPR) on their ability to reconstruct the channel response and support downstream bit detection.

Identify applicable funding agency here. If none, delete this.

II. Related Works

III. Proposed Methodology

A. Dataset Generation

Because there was no publicly accessible dataset for the mentioned regression task on diffusion-based molecular communication, a synthetic dataset was generated using the analytical diffusion-based molecular communication channel model. The objective of the data set creation was to emulate stochastic nature of molecular propagation under different communication channel conditions and to provide a reliable benchmark for training and evaluating regression models.

The dataset consists of **75,000** different molecular communication cases. Each case is developed through the random selection of the parameters within practical operational limits for each case. These parameters comprises transmitter-receiver distance, receiver radius, diffusion coefficient, number of released molecules, bit duration, previous transmitted bit, current transmitted bit, environmental temperature, and flow velocity. Randomizing these parameters allows the dataset to represent a wide variety of molecular communication environments instead of a single fixed channel configuration.

In each of the generated cases, the volume of the receiver was determined under the assumption that the receiving end is spherical and absorptive. The Channel Impulse Response

(CIR) was computed through the diffusion equation, which models the probability of molecules reaching the receiver over time. Rather than selecting an arbitrary observation instant, the peak value of the CIR and its corresponding arrival time were determined, as these represent the maximum expected molecular concentration at the receiver.

The expected number of received molecules was obtained by multiplying the peak(maximum) CIR with the number of molecules transmitted from the transmitter. As the movement of the molecules is a random process due to the Brownian motion phenomenon, the number of received molecules was simulated using a Poisson distribution, which accurately models the stochastic arrival process of molecules at the receiver. In order to make the generated data more realistic and to simulate the real channel uncertainties, additive Gaussian noise was added to the received molecules.

The generated data was stored in CSV files, where each row corresponds to a unique molecular communication scenario. The resulting dataset was subsequently used to train and evaluate the proposed regression models for predicting the number of molecules received under varying channel conditions.

B. Feature Engineering

We applied Four feature engineering steps prior to model training.

Physical validity filtering. Any realization where Received_Molecules $\leq$ Molecules_Released were discarded,since this violates basic physical constraints as described in Section III-A.

Leakage control. We excluded Expected_Molecules from the model inputs— the theoretically received count is computed analytically from the closed-form CIR expression — Because this feature is derived from the same physical relationship that generates the target variable and as a result correlates almost perfectly with Received$_Molecules$(r ≈ 0.9998) in the cleaned data. Keping it as it is ,would let a model to shortcut the actual learning task, mainly just recovering an already known analytical mapping inspite of inferring the channel-to-reception relationship from data. We kept Expected_Molecules retained only as an analytical baseline for comparison against the trained models.

Target transformation. The target variable exhibits pronounced right-skew (skewness $\approx$ 6.6), with approximately 35% of realizations registering zero received molecules. A $\log(1+x)$ transformation was applied to Received_Molecules prior to training,

$$y' = \log(1 + y) \tag{1}$$

with model predictions inverse-transformed via $y = \exp(y') - 1$ before evaluation. This step was found to be necessary for kernel-based methods such as Support Vector Regression, which are sensitive to the scale and skew of the target variable.

Correlation-based feature review and scaling. Pairwise Pearson correlations between each input feature and the target variable were computed to characterize the relative influence of channel parameters on reception. All input features were standardized (zero mean, unit variance) prior to training to ensure comparability across models with different scale sensitivities.

C. Regression Models

Ten regression models spanning linear, kernel-based, instance-based, tree-based, and neural network approaches were trained and compared. For each model, hyperparameters were selected via 5-fold cross-validated grid search over the ranges listed below, optimizing for R^2 score on the training set.

- **Linear Regression (LR).** Models the target as a linear combination of the input features. No hyperparameters were tuned; LR serves as a baseline for assessing the extent to which reception is linearly related to the input channel parameters.
- **Ridge Regression.** Linear regression with an L_2 penalty on the coefficients to reduce overfitting. Regularization strength α was searched over $\{0.01, 0.1, 1, 10, 100\}$.
- **Lasso Regression.** Linear regression with an L_1 penalty, which additionally performs feature selection by shrinking less relevant coefficients to zero. α was searched over $\{0.0001, 0.001, 0.01, 0.1, 1\}$.
- **Elastic Net.** Combines L_1 and L_2 penalties. α was searched over $\{0.001, 0.01, 0.1, 1\}$ and the L_1/L_2 mixing ratio (`l1_ratio`) over $\{0.2, 0.5, 0.8\}$.
- **Support Vector Regressor (SVR).** Fits a regression function with an RBF kernel, trained on the log-transformed target to address the scale sensitivity noted in Section III-B. Hyperparameters searched: penalty parameter $C \in \{1, 10, 100\}$, kernel coefficient `gamma` $\in \{\text{scale}, \text{auto}\}$, and epsilon (error tolerance margin) $\in \{0.01, 0.05, 0.1\}$.
- **K-Neighbor Regressor (KNR).** Predicts the output as the average target value among the k nearest neighbors in the training data. Number of neighbors k was searched over $\{3, 5, 7, 9\}$, with weighting scheme over $\{\text{uniform}, \text{distance}\}$.
- **Decision Tree Regressor (DTR).** Partitions the input space through threshold-based splits. Maximum tree depth was searched over $\{5, 10, 15, 20, \text{unlimited}\}$, and minimum samples required to split a node over $\{2, 5, 10\}$.
- **Random Forest Regressor (RFR).** An ensemble of decision trees trained on bootstrap samples with random feature subsets at each split. Number of trees was searched over $\{100, 200, 300\}$ and maximum depth over $\{10, 20, \text{unlimited}\}$.
- **Gradient Boosting Regressor (GBR).** Builds an additive ensemble of weak learners sequentially, each trained to reduce the residual error of the combined model. Number of estimators was searched over $\{100, 200\}$, learning rate over $\{0.01, 0.05, 0.1\}$, and maximum depth over $\{3, 5\}$.
- **XGBoost Regressor.** A regularized, gradient-boosted tree ensemble. Number of estimators was searched over

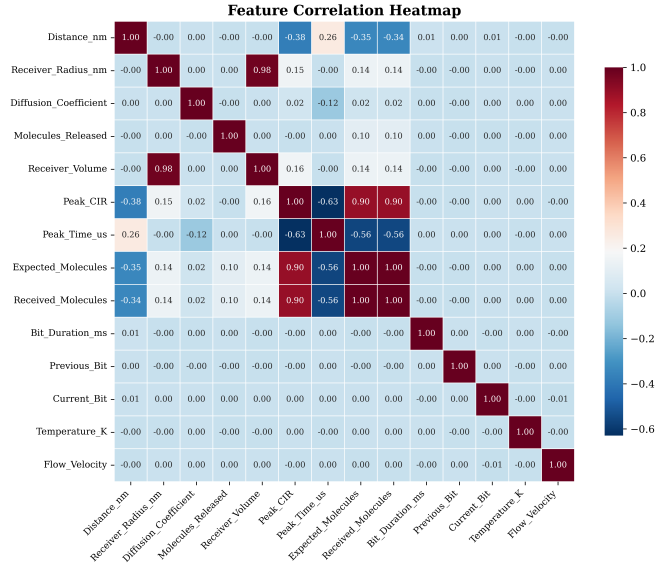


Fig. 1. Pearson correlation matrix of molecular communication parameters.

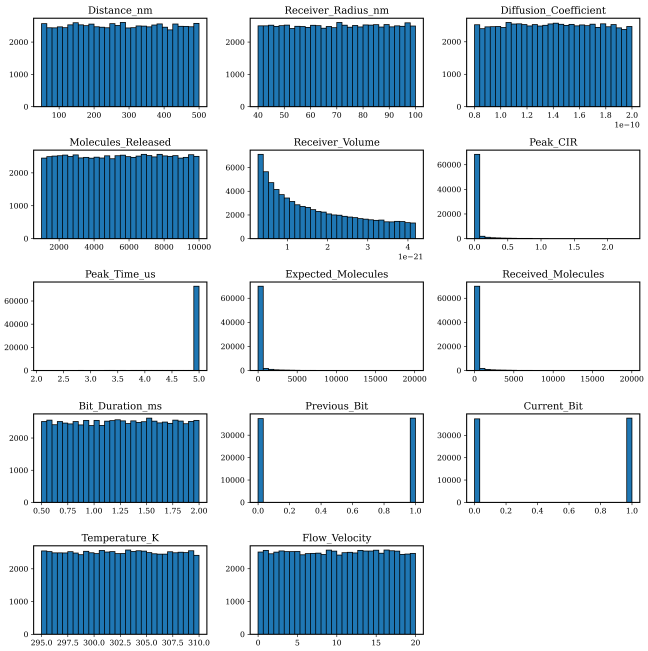


Fig. 2. Distribution of input features after preprocessing.

$\{100, 200\}$, learning rate over $\{0.05, 0.1\}$, and maximum depth over $\{3, 5, 7\}$.

For each model, the best-performing hyperparameter configuration identified via cross-validation is reported in Section IV alongside the corresponding test-set performance metrics.

IV. Results and Discussions

A. Experimental Results

All regression models here are implemented in python using Scikit-learn library, while XGBoost was implemented using the XGBoost framework. here the cleaned dataset consisting of 75,000 realizations and divided into training and testing subsets using an 80:20 split. The Model performance was evaluted using Mean Absolute Error(MAE), Rot Mean Square Error(RMSE), cofficient of determination(R^2), training time, and prediction time. Table I summarizes the performance of all evaluated regression models. Ensemble learning approaches achieved the highest prediction accuracy, whereas linear regression models produced comparatively lower R^2 scores because of the nonlinear behavior of molecular diffusion. Among all models, the best-performing model achieved the highest coefficient of determination while maintaining low prediction error and acceptable computational cost.

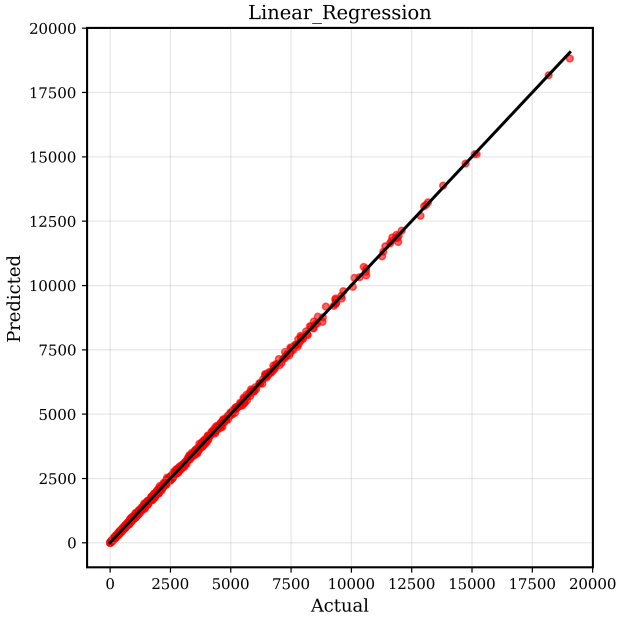


Fig. 3. Actual versus predicted values for Linear Regression.

B. Correlation and Feature Analysis

C. Model-wise Prediction Analysis

1) *Linear Regression:*
2) *Polynomial Regression:*
3) *Artificial Neural Network:*
4) *Decision Tree:*
5) *Gradient Boosting:*
6) *XGBoost:*
7) *Transformer:*

D. Feature Importance and Explainability

E. Error Analysis

F. Performance Comparison

G. Bit Error Rate Analysis

H. Overall Ranking

The obtained results confirm the possibility of using machine learning models for approximating molecular communication channel responses. The results show that ensemble

TABLE I
PERFORMANCE COMPARISON OF THE EVALUATED REGRESSION MODELS.

Model	**Architecture**	**MAE**	**RMSE**	R^2	**Train Time (s)**	**Pred. Time (s)**
Linear Regression	Linear	**5.4379**	**15.1227**	**0.999758**	**0.0756**	0.0061
Polynomial Regression (Deg. 2)	Degree = 2	5.4584	15.2391	0.999754	1.0718	0.0506
Polynomial Regression (Deg. 3)	Degree = 3	5.4667	15.1575	0.999757	10.2597	0.1142
ANN	[128, 64]	6.4189	17.4987	0.999676	65.5619	1.3222
Gradient Boosting	100 Trees	6.1649	18.7422	0.999629	51.7650	0.0149
Decision Tree	Depth = 10	6.4310	20.0896	0.999573	1.2100	**0.0030**
XGBoost	Boosted Trees	11.7315	64.6497	0.995580	30.3475	0.0406
Transformer	Best Architecture	88.0988	449.2948	0.786539	684.6869	4.0204

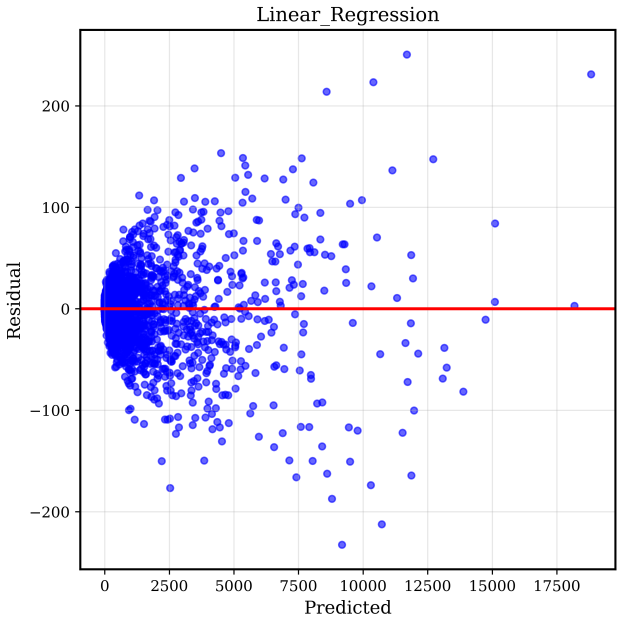


Fig. 4. Residual analysis of Linear Regression.

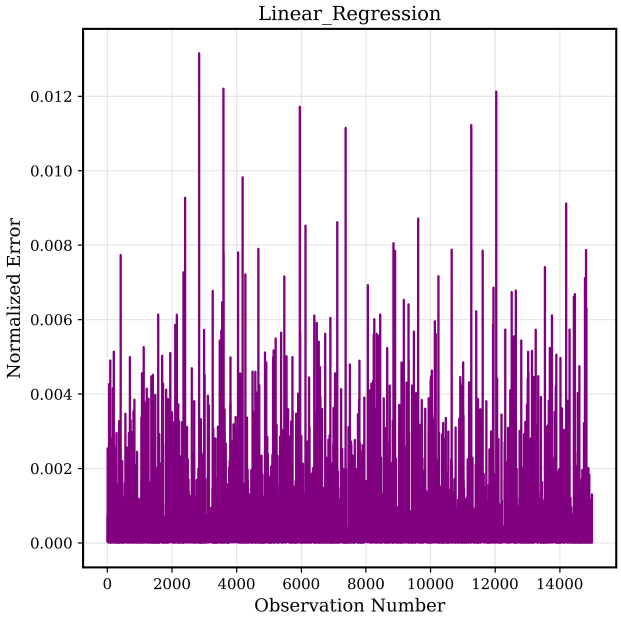


Fig. 5. Normalized prediction error of Linear Regression.

models allow for achieving high prediction accuracy, while the use of simpler models is justified by their lower computational complexity. The importance of different features and their impact on the accuracy of models were also studied.

Figure **??** summarizes the overall ranking of the evaluated regression models considering prediction accuracy and computational performance. Ensemble-based models consistently outperform conventional linear approaches by effectively capturing the nonlinear characteristics of molecular diffusion channels.

The study shows that the number of received molecules in a MC channel can be successfully estimated using machine learning regression approaches. In particular, the analysis demonstrates that ensemble approaches provide an optimal compromise between accuracy and complexity, which is of particular interest for the design of realistic molecular communication systems.

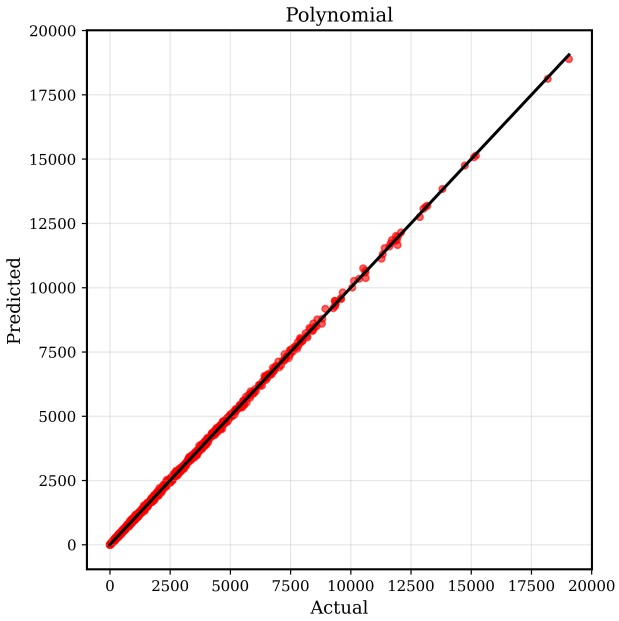


Fig. 6. Actual versus predicted values for Polynomial Regression.

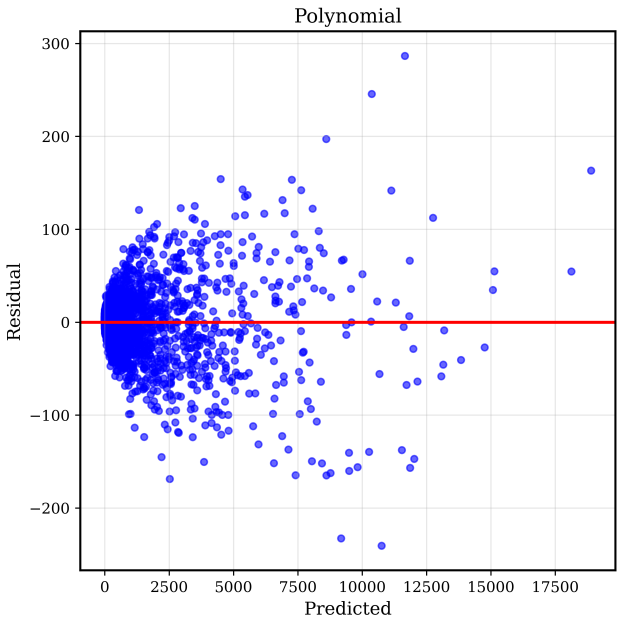


Fig. 7. Residual analysis of Polynomial Regression.

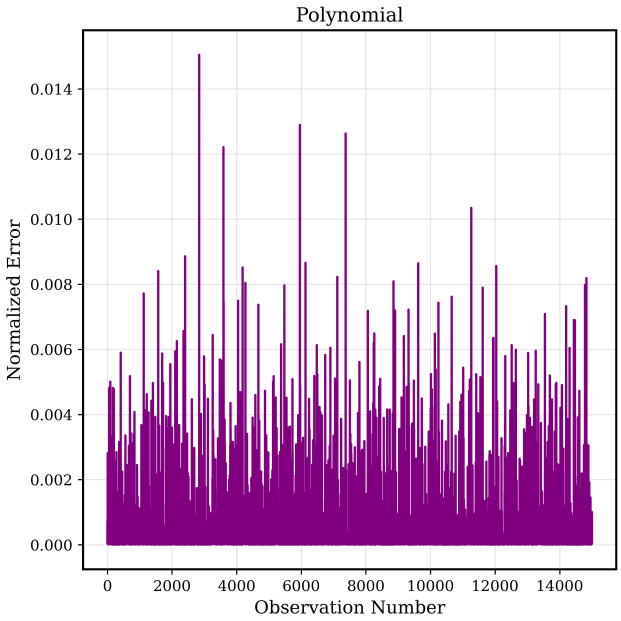


Fig. 8. Normalized error analysis of Polynomial Regression.

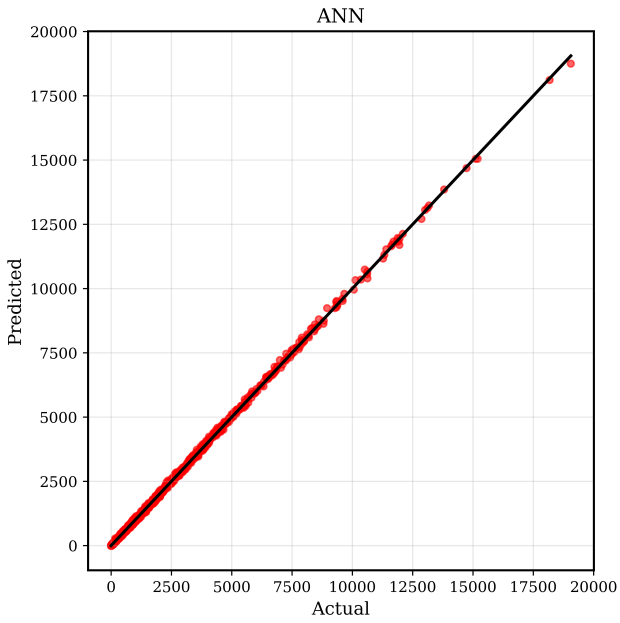


Fig. 9. Actual versus predicted values for ANN.

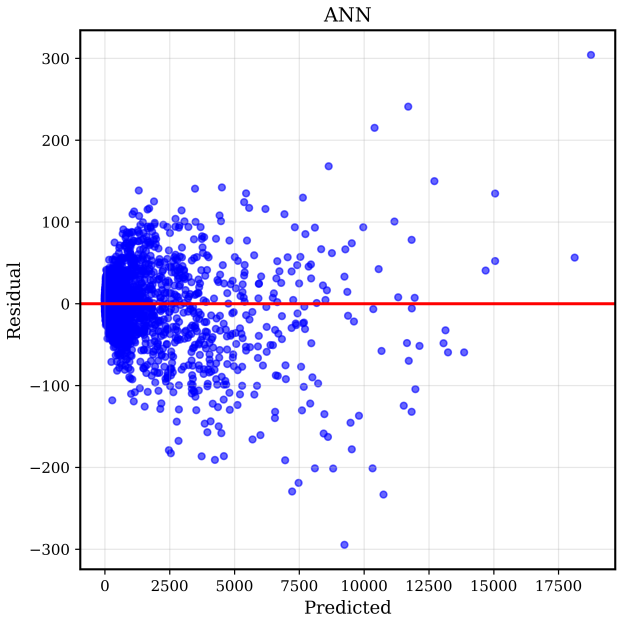


Fig. 10. Residual analysis of ANN.

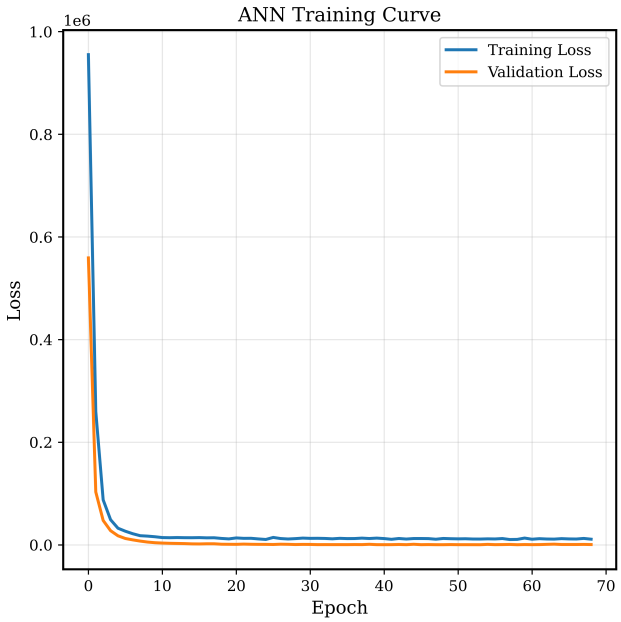


Fig. 12. Training loss convergence of ANN.

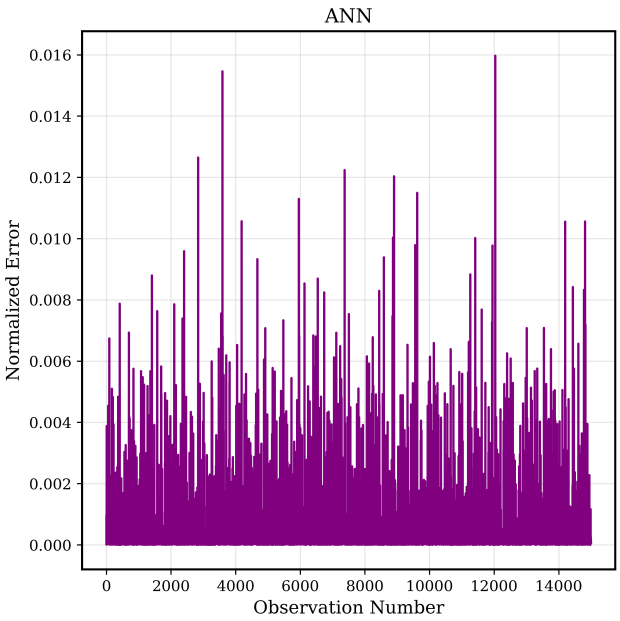


Fig. 11. Normalized prediction error of ANN.

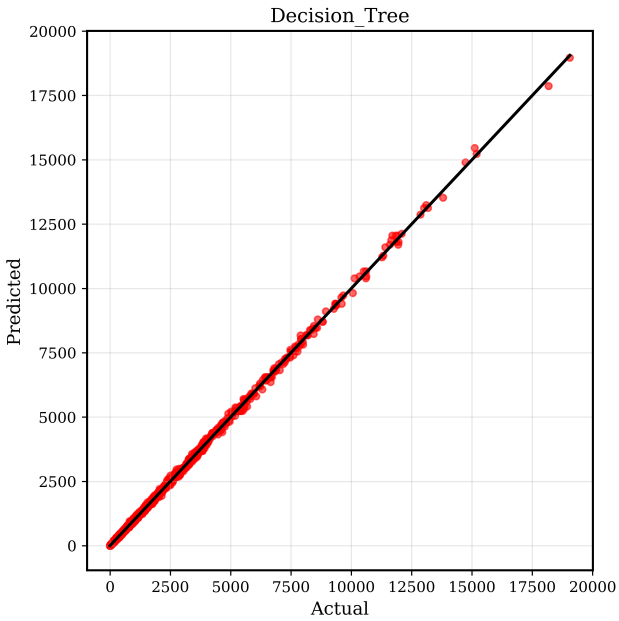


Fig. 13. Actual versus predicted values for Decision Tree.

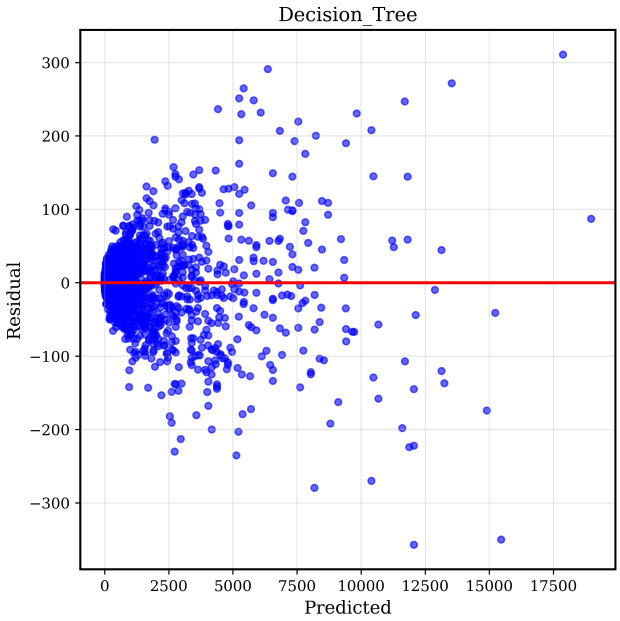


Fig. 14. Residual analysis of Decision Tree.

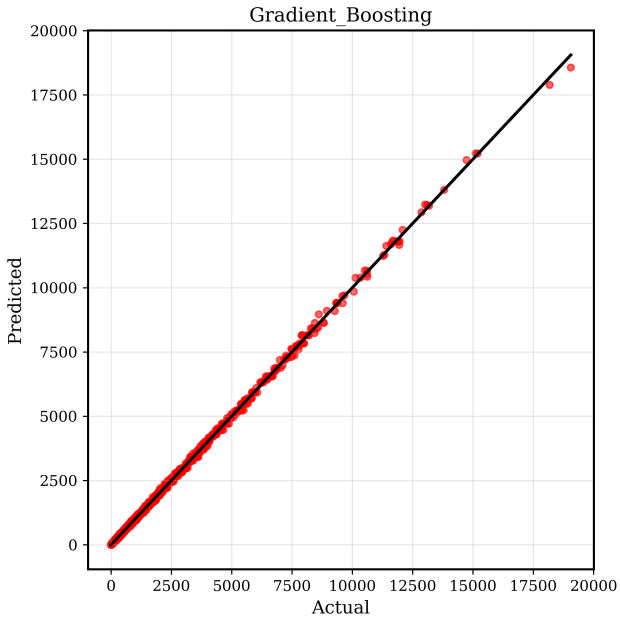


Fig. 16. Actual versus predicted values for Gradient Boosting.

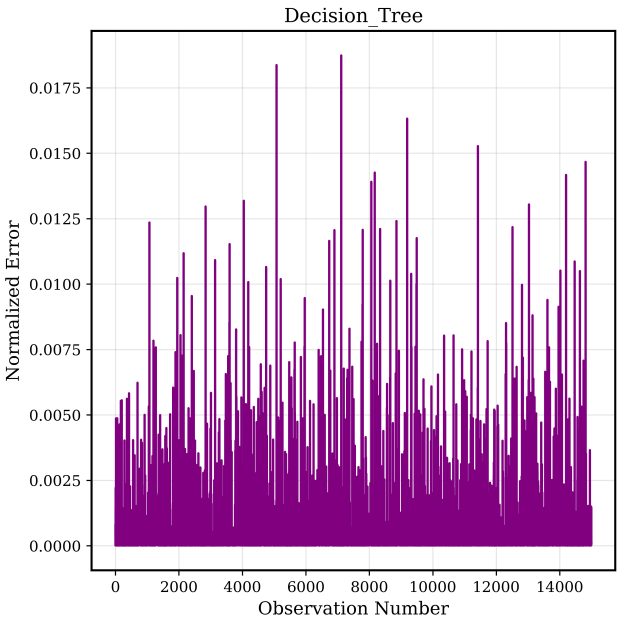


Fig. 15. Normalized error of Decision Tree.

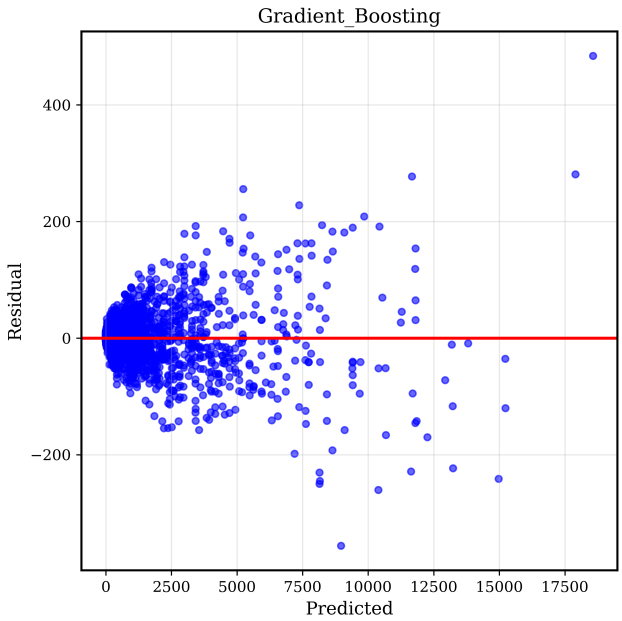


Fig. 17. Residual analysis of Gradient Boosting.

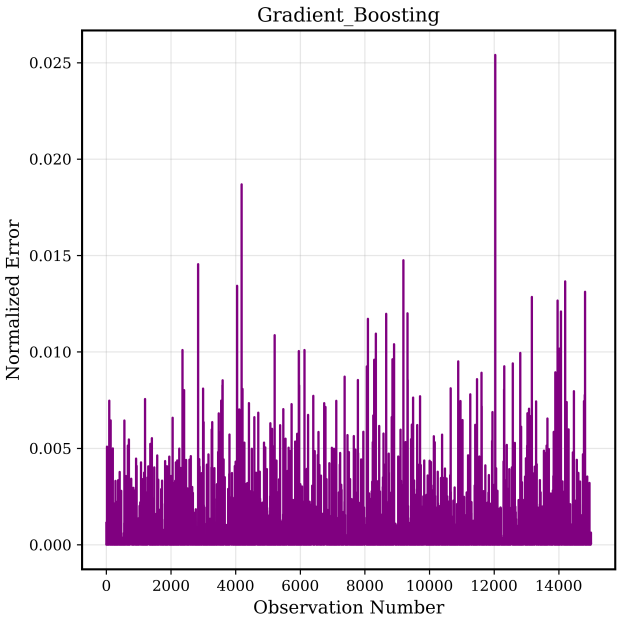


Fig. 18. Normalized error analysis of Gradient Boosting.

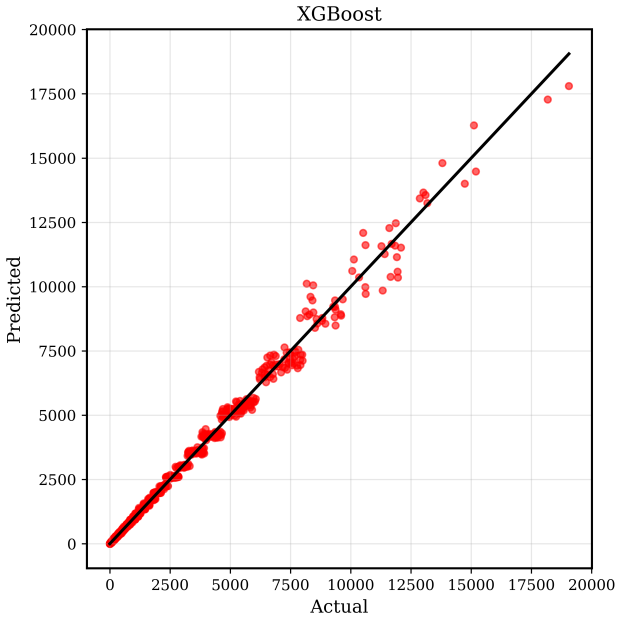


Fig. 19. Actual versus predicted values for XGBoost.

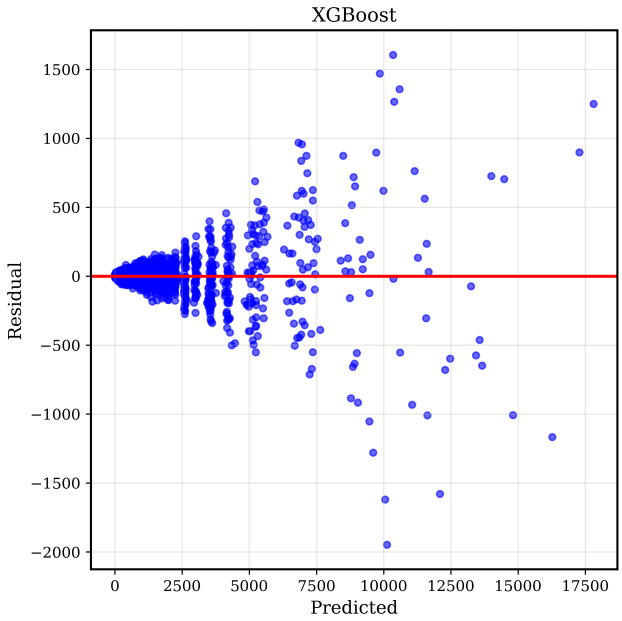


Fig. 20. Residual analysis of XGBoost.

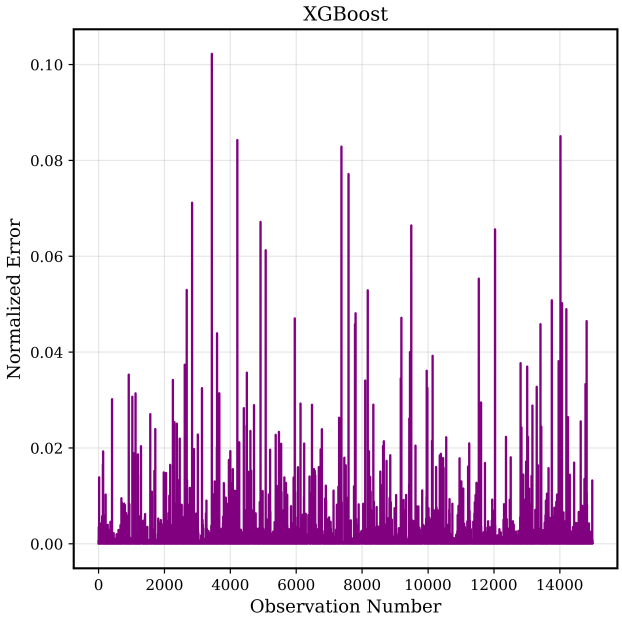


Fig. 21. Normalized error analysis of XGBoost.

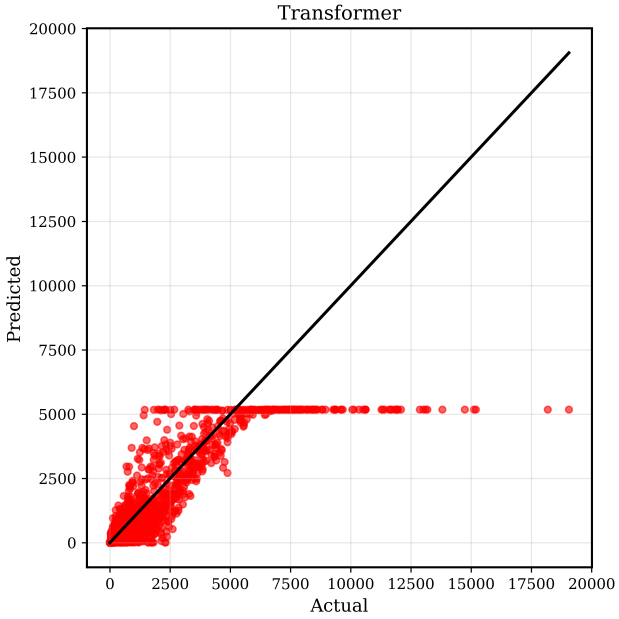


Fig. 22. Actual versus predicted values for Transformer model.

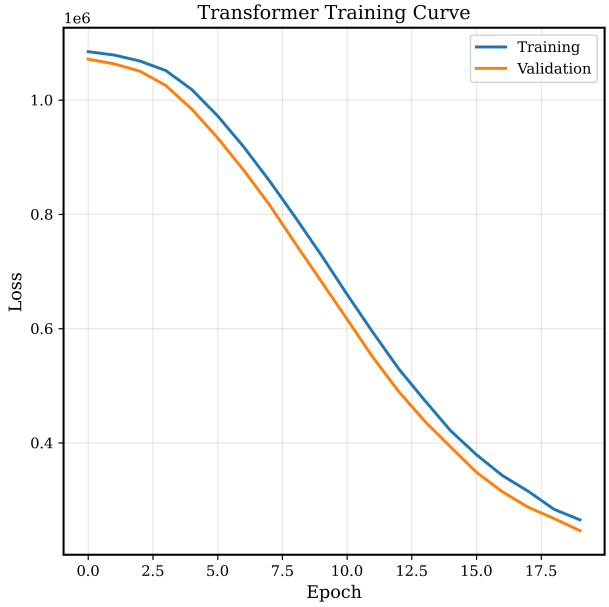


Fig. 24. Training loss curve of Transformer.

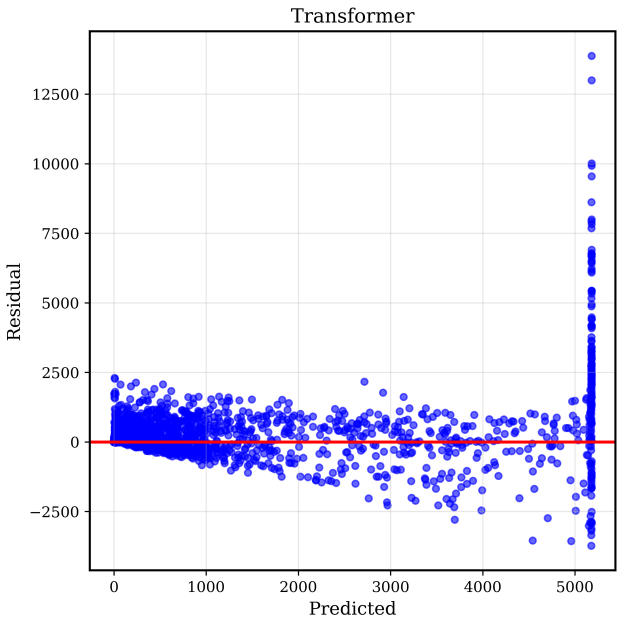


Fig. 23. Residual analysis of Transformer.

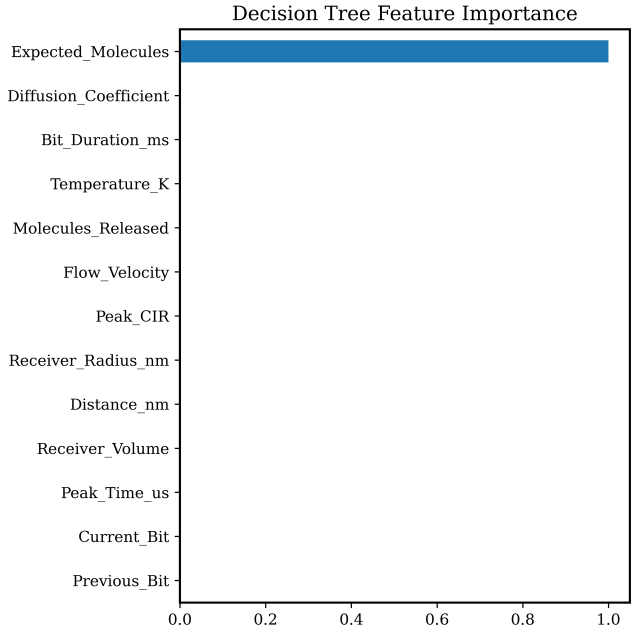


Fig. 25. Feature importance obtained from Decision Tree.

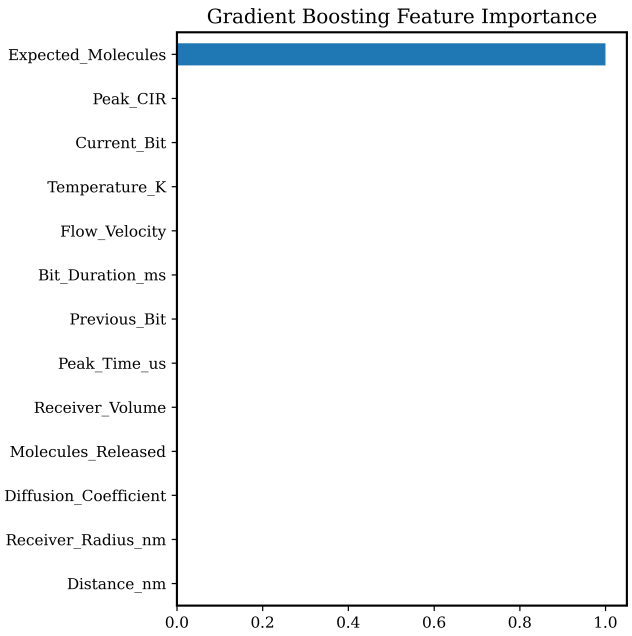


Fig. 26. Feature importance obtained from Gradient Boosting.

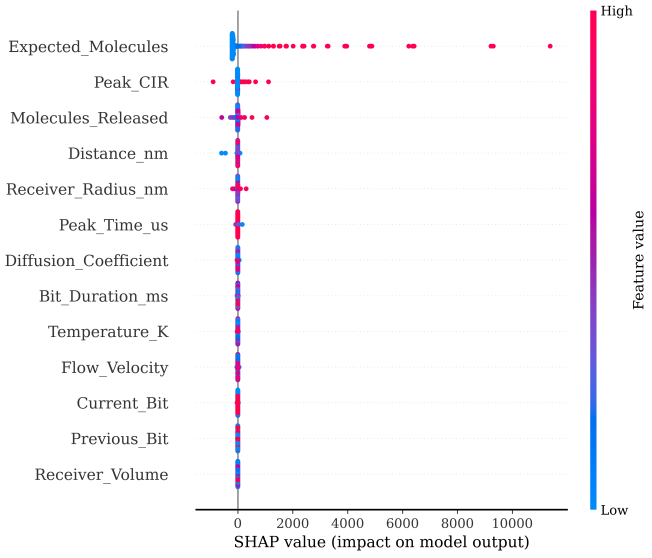


Fig. 27. SHAP-based explanation of feature contribution.

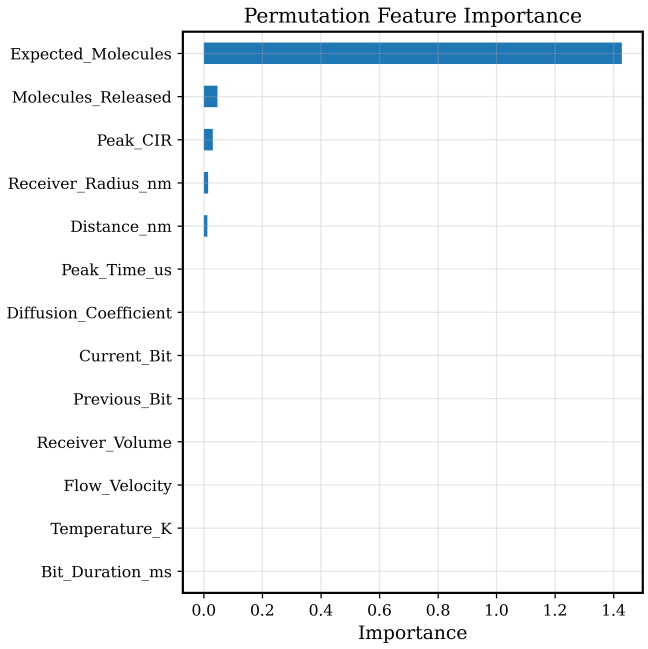


Fig. 28. Permutation feature importance analysis.

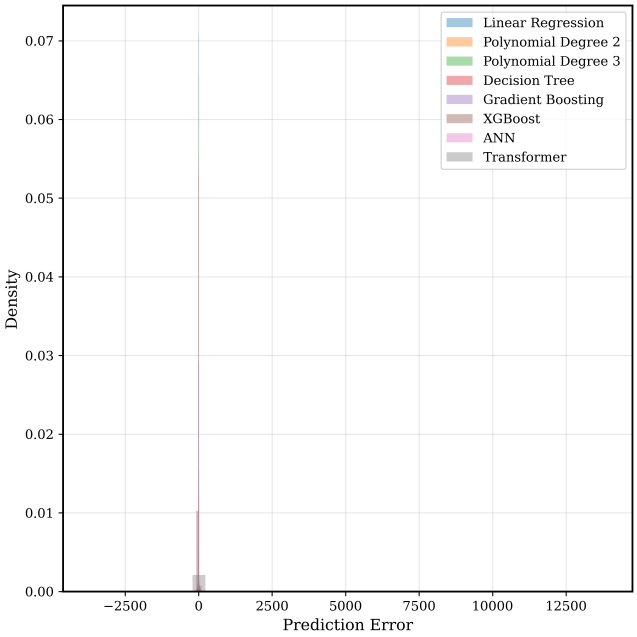


Fig. 29. Distribution of prediction errors.

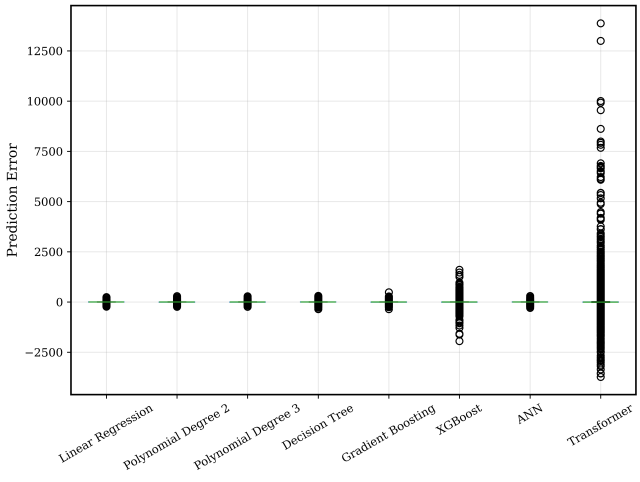


Fig. 30. Comparison of error distributions among models.

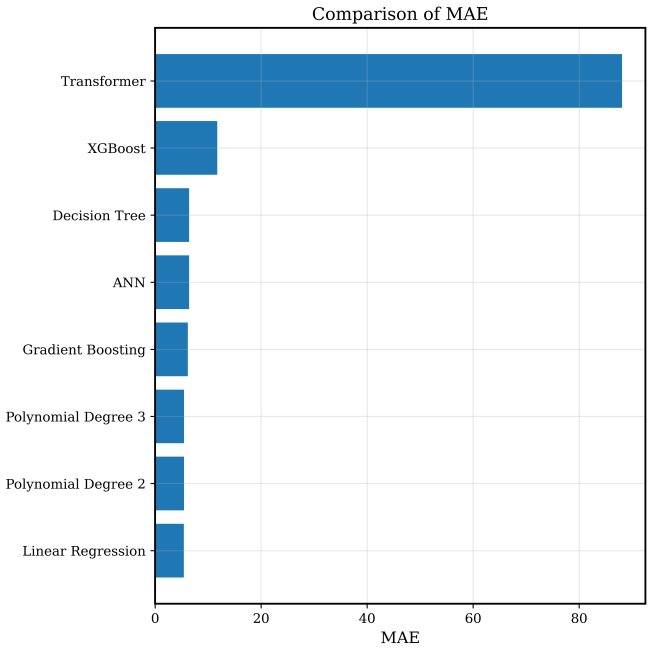


Fig. 31. MAE comparison of regression models.

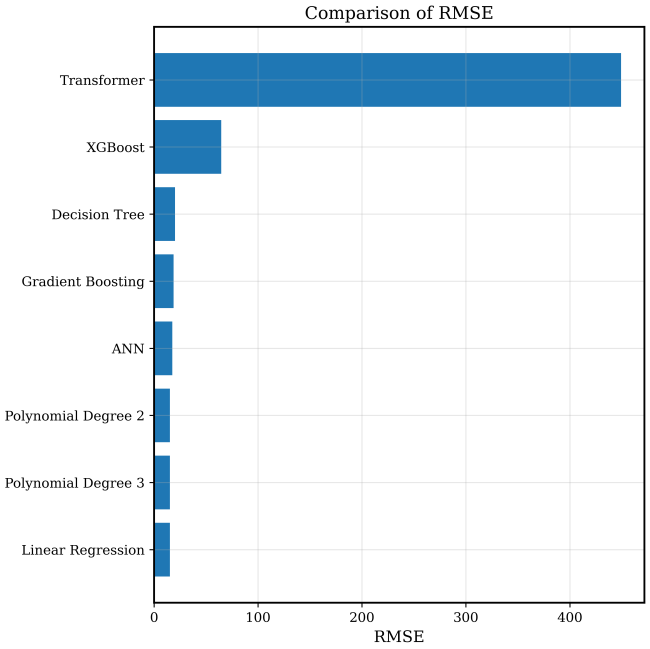


Fig. 32. RMSE comparison of regression models.

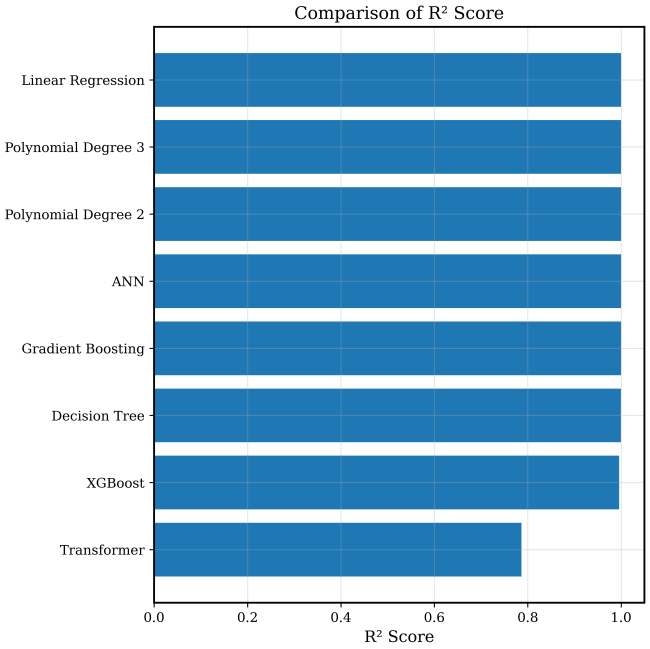


Fig. 33. R^2 comparison of regression models.

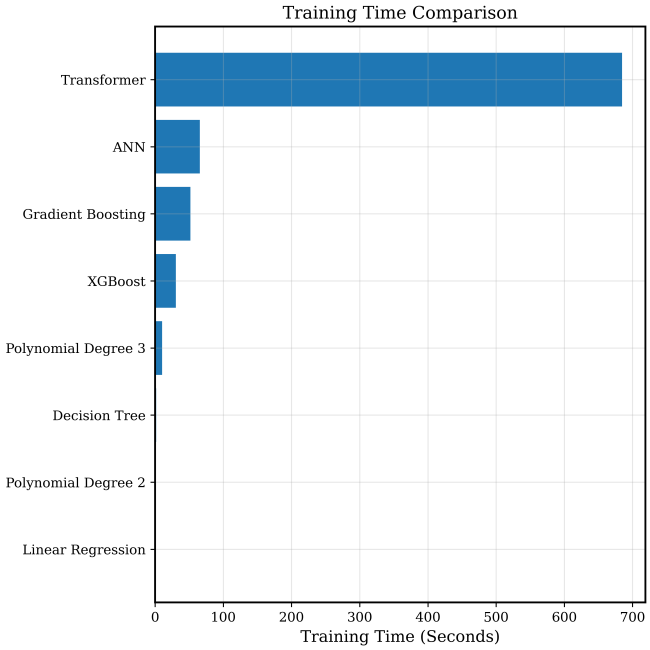


Fig. 34. Training time comparison.

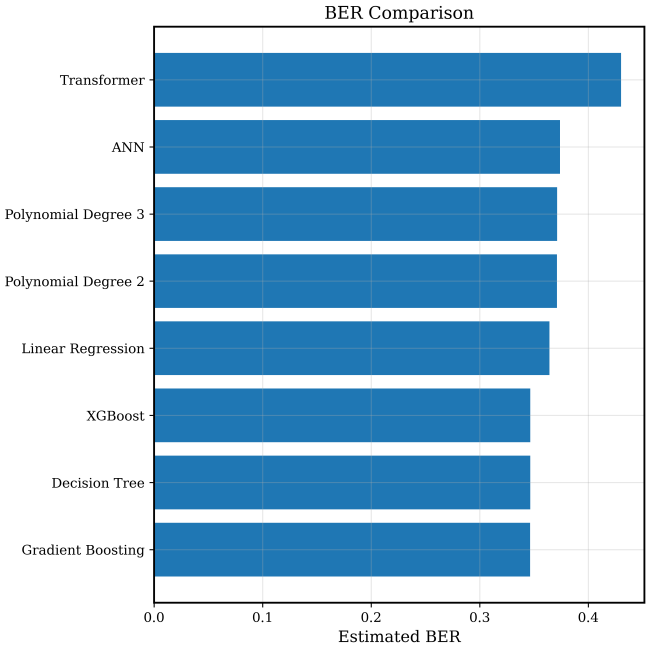


Fig. 36. BER comparison among regression models.

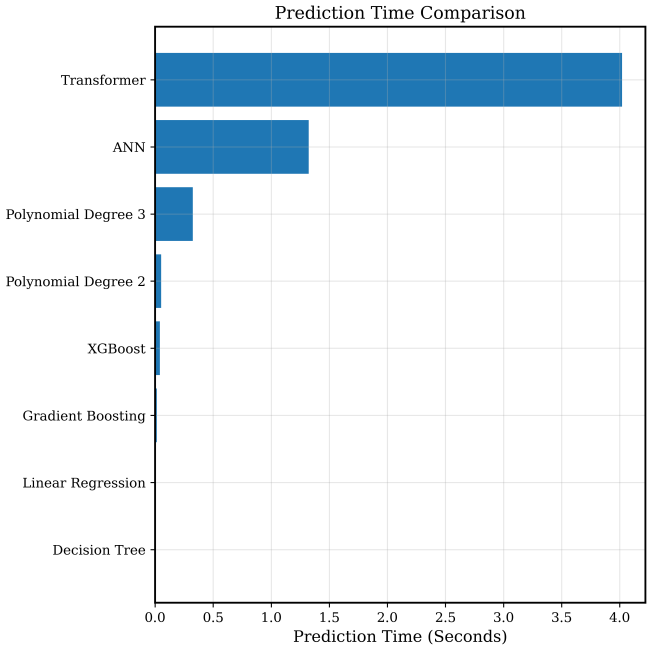


Fig. 35. Prediction time comparison.

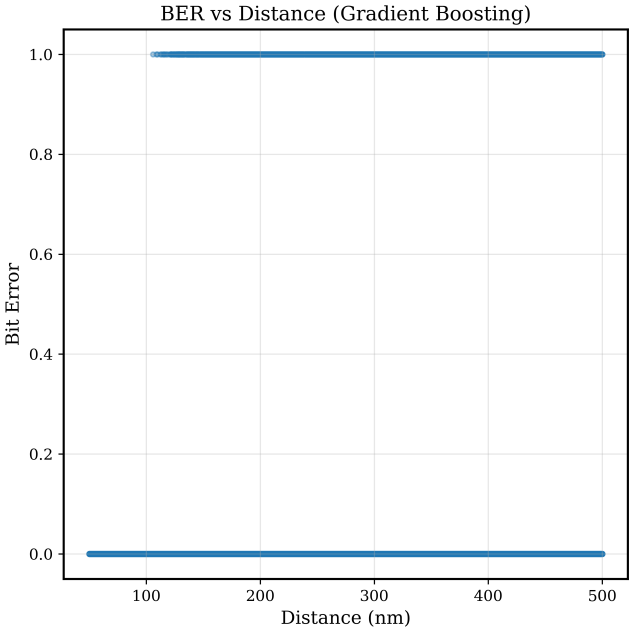


Fig. 37. Variation of BER with transmitter-receiver distance.

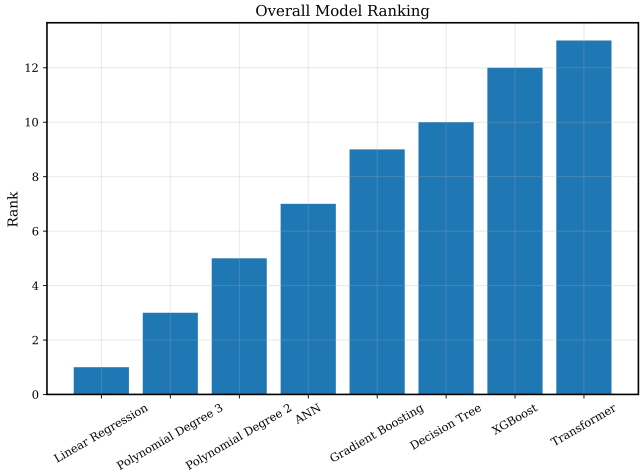


Fig. 38. Overall ranking of regression models.

REFERENCES

[1] J. T. Gómez, P. Hofmann, F. H. Fitzek, and F. Dressler, "Explainability of neural networks for symbol detection in molecular communication channels," *IEEE Transactions on Molecular, Biological, and Multi-Scale Communications*, vol. 9, no. 3, pp. 323–328, 2023.

[2] Y. Huang, F. Ji, Z. Wei, M. Wen, and W. Guo, "Signal detection for molecular communication: Model-based vs. data-driven methods," *IEEE communications magazine*, vol. 59, no. 5, pp. 47–53, 2021.